

SETUP AND ACCESS

Configure server-managed settings

Centrally configure Claude Code for your organization through server-delivered settings, without requiring device management infrastructure.

Server-managed settings let organization Owners centrally configure Claude Code from **Admin Settings > Claude Code > Managed settings** in the claude.ai console. Claude Code clients fetch these settings automatically when users authenticate with an organization OAuth login or a directly configured API key, on platforms where server-managed delivery is supported. See **Platform availability**.

This approach is designed for organizations that do not have device management infrastructure in place, or that need to manage settings for users on unmanaged devices.

❗ Server-managed settings are available for [Claude for Teams](#) and [Claude for Enterprise](#) customers.

Requirements

To use server-managed settings, you need:

- Claude for Teams or Claude for Enterprise plan
- The Owner or Primary Owner role in your Claude organization, to view and edit the configuration
- Claude Code version 2.1.38 or later for Claude for Teams, or version 2.1.30 or later for Claude for Enterprise
- Network access to `api.anthropic.com`

Choose between server-managed and endpoint-managed settings

Claude Code supports two approaches for centralized configuration. Server-managed settings deliver configuration from Anthropic's servers. **Endpoint-managed settings** are deployed directly to devices through native OS policies (macOS managed preferences, Windows registry) or managed settings files.

Approach	Best for	Security model
Server-managed settings	Organizations without MDM, or users on unmanaged devices	Settings delivered from Anthropic's servers at authentication time
<u>Endpoint-managed settings</u>	Organizations with MDM or endpoint management	Settings deployed to devices via MDM configuration profiles, registry policies, or managed settings files

If your devices are enrolled in an MDM or endpoint management solution, endpoint-managed settings provide stronger security guarantees because the settings file can be protected from user modification at the OS level. Endpoint-managed settings do not reach cloud sessions, so organizations using Claude Code on the web should configure server-managed settings as well.

Configure server-managed settings

1 Open the admin console

In the claude.ai console, go to Admin Settings > Claude Code > Managed settings.

If the link redirects you to a different Admin Settings page instead of the Claude Code page, your account doesn't have the required role. Admin and other non-Owner roles can't view or edit managed settings, so ask an Owner or Primary Owner in your organization to make the change. See Access control.

2 Define your settings

Add your configuration as JSON. All settings available in settings.json are supported except those restricted to OS-level policy delivery; see Current limitations for that short list. This includes hooks, environment variables, and managed-only settings like `allowManagedPermissionRulesOnly`.

This example enforces a permission deny list, prevents users from bypassing permissions, and restricts permission rules to those defined in managed settings:

```
{
  "permissions": {
    "deny": [
      "Bash(curl *)",
      "Read(../.env)",
      "Read(../.env.*)",
      "Read(../secrets/**)"
    ],
    "disableBypassPermissionsMode": "disable"
  },
  "allowManagedPermissionRulesOnly": true
}
```

Hooks use the same format as in `settings.json` .

This example runs an audit script after every file edit across the organization:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          { "type": "command", "command":
"/usr/local/bin/audit-edit.sh" }
        ]
      }
    ]
  }
}
```

To configure the **auto mode** classifier so it knows which repos, buckets, and domains your organization trusts:

```
{
  "autoMode": {
    "environment": [
      "Source control: github.example.com/acme-corp and all
repos under it",
      "Trusted cloud buckets: s3://acme-build-artifacts,
gs://acme-ml-datasets",
      "Trusted internal domains: *.corp.example.com"
    ]
  }
}
```

Because hooks execute shell commands, users see a **security approval dialog** before they're applied. See **Configure auto mode** for how the `autoMode` entries affect what the classifier blocks and important warnings about the `environment` , `allow` , `soft_deny` , and `hard_deny` fields.

3 Save and deploy

Save your changes. Claude Code clients receive the updated settings on their next startup or hourly polling cycle.

Verify settings delivery

To confirm that settings are being applied, ask a user to restart Claude Code. If the configuration includes settings that trigger the **security approval dialog**, the user sees a prompt describing the managed settings on startup. You can also verify that managed permission rules are active by having a user run `/permissions` to view their effective permission rules.

Access control

The following roles can manage server-managed settings:

- **Primary Owner**
- **Owner**

Restrict access to trusted personnel, as settings changes apply to all users in the organization.

Managed-only settings

Most **settings keys** work in any scope. A handful of keys are only read from managed settings and have no effect when placed in user or project settings files. See **managed-only settings** for the full list. Any setting not on that list can still be placed in managed settings and takes the highest precedence.

Current limitations

Server-managed settings have the following limitations:

- Settings apply uniformly to all users in the organization. Per-group configurations are not yet supported.
- A `managed-mcp.json` file cannot be distributed through server-managed settings. Deliver the `allowedMcpServers` and `deniedMcpServers` policy keys there instead.
- Settings restricted to OS-level policy sources, such as `policyHelper` and `wsInheritsWindowsSettings`, are not honored. Deploy them through MDM or a system `managed-settings.json` file instead.

Settings delivery

Settings precedence

Server-managed settings and **endpoint-managed settings** both occupy the highest tier in the Claude Code **settings hierarchy**. No other settings level can override them, including command line arguments.

Within the managed tier, the first source that delivers a non-empty configuration wins. Server-managed settings are checked first, then endpoint-managed settings. Sources do not merge: if server-managed settings deliver any keys at all, endpoint-managed settings are ignored entirely. If server-managed settings deliver nothing, endpoint-managed settings apply.

If you clear your server-managed configuration in the admin console with the intent of falling back to an endpoint-managed plist or registry policy, be aware that **cached settings** persist on client machines until the next successful fetch. Run `/status` to see which managed source is active.

Fetch and caching behavior

Claude Code fetches settings from Anthropic's servers at startup and polls for updates hourly during

active sessions.

First launch without cached settings:

- Claude Code fetches settings asynchronously
- If the fetch fails, Claude Code continues without managed settings
- There is a brief window before settings load where restrictions are not yet enforced

Subsequent launches with cached settings:

- Cached settings apply immediately at startup
- Claude Code fetches fresh settings in the background
- Cached settings persist through network failures

Claude Code applies settings updates automatically without a restart, except for advanced settings like OpenTelemetry configuration, which require a full restart to take effect.

Invalid entries in delivered settings

Delivered payloads parse tolerantly with the same rules as the other managed sources. When a payload contains an entry that fails schema validation, Claude Code strips that entry, surfaces a validation error, and applies every remaining valid setting. See [Invalid entries in managed settings](#) for the field-level behavior, including how security-enforcement fields are handled. Requires Claude Code v2.1.169 or later.

Server-managed delivery adds these behaviors:

- The cache at `~/.claude/remote-settings.json` stores the salvaged payload with invalid entries removed. The raw invalid payload is never persisted.
- When no field in the payload can be salvaged, Claude Code keeps the last-accepted cached settings and records a fatal error.
- The [security approval dialog](#) evaluates the salvaged payload, so a stripped invalid entry is never presented for approval and never executes.

To debug delivery issues, run `claude --debug-file <path>` and search the log for `Remote settings`. Validate a payload change with `claude doctor` on a test machine before rolling it out to the organization.

Enforce fail-closed startup

By default, if the remote settings fetch fails at startup, the CLI continues without managed settings. For environments where this brief unenforced window is unacceptable, set `forceRemoteSettingsRefresh: true` in your managed settings.

When this setting is active, the CLI blocks at startup until remote settings are freshly fetched. If the fetch fails, the CLI exits rather than proceeding without the policy. This setting self-perpetuates: once delivered from the server, it is also cached locally so that subsequent startups enforce the same behavior even before the first successful fetch of a new session.

To enable this, add the key to your managed settings configuration:

```
{
  "forceRemoteSettingsRefresh": true
}
```

You can also set this key in an endpoint-managed MDM profile or system `managed-settings.json` file to enforce fail-closed behavior on first launch, before any server payload has been delivered. As of v2.1.191, this flag is an exception to the precedence rule above: it is honored when set in any managed source even if a cached server-managed payload is also present, so an MDM-delivered value is not ignored when server-managed settings exist. The settings fetch also sends a `Cache-Control: no-cache` header so intermediate HTTP proxies do not serve a stale response.

Before enabling this setting, ensure your network policies allow connectivity to `api.anthropic.com`. If that endpoint is unreachable, the CLI exits at startup and users cannot start Claude Code.

As of v2.1.139, the `claude auth` subcommands such as `claude auth login` are exempt from this check, so users can re-authenticate when expired credentials are the reason the settings fetch fails.

Security approval dialogs

Certain settings that could pose security risks require explicit user approval before being applied:

- **Shell command settings:** settings that execute shell commands
- **Custom environment variables:** variables not in the known safe allowlist
- **Hook configurations:** any hook definition

When these settings are present, users see a security dialog explaining what is being configured. Users must approve to proceed. If a user rejects the settings, Claude Code exits.

 In non-interactive mode with the `-p` flag, Claude Code skips security dialogs and applies settings without user approval.

Platform availability

Server-managed settings require a direct connection to `api.anthropic.com` , and delivery requires the session to authenticate with an organization OAuth login or a directly configured API key: keys returned by an `apiKeyHelper` script do not trigger the settings fetch. Server-managed settings are not available when using third-party model providers:

- Amazon Bedrock
- Google Vertex AI
- Microsoft Foundry
- **Claude Platform on AWS**
- Custom API endpoints via `ANTHROPIC_BASE_URL` or **LLM gateways**

Audit logging

Audit log events for settings changes are available through the compliance API or audit log export. Contact your Anthropic account team for access.

Audit events include the type of action performed, the account and device that performed the action, and references to the previous and new values.

Security considerations

Server-managed settings provide centralized policy enforcement, but they operate as a client-side control. On unmanaged devices, users with admin or sudo access can modify the Claude Code binary, filesystem, or network configuration.

Scenario	Behavior
User edits the cached settings file	Tampered file applies at startup, but correct settings restore on the next server fetch

Scenario	Behavior
User deletes the cached settings file	First-launch behavior occurs: settings fetch asynchronously with a brief unenforced window
API is unavailable	Cached settings apply if available, otherwise managed settings are not enforced until the next successful fetch. With <code>forceRemoteSettingsRefresh: true</code> , the CLI exits instead of continuing, except for <u><code>claude auth subcommands</code></u>
User authenticates with a different organization	Settings are not delivered for accounts outside the managed organization
User configures a <u>third-party model provider</u>	Server-managed settings are bypassed. This includes setting <code>CLAUDE_CODE_USE_BEDROCK</code> , <code>CLAUDE_CODE_USE_MANTLE</code> , <code>CLAUDE_CODE_USE_VERTEX</code> , <code>CLAUDE_CODE_USE_FOUNDRY</code> , <code>CLAUDE_CODE_USE_ANTHROPIC_AWS</code> , or a non-default <code>ANTHROPIC_BASE_URL</code>

To detect runtime configuration changes, use `ConfigChange hooks` to log modifications or block unauthorized changes before they take effect.

For stronger enforcement guarantees, use `endpoint-managed settings` on devices enrolled in an MDM solution.

See also

Related pages for managing Claude Code configuration:

- **Settings**: complete configuration reference including all available settings
- **Endpoint-managed settings**: managed settings deployed to devices by IT
- **Authentication**: set up user access to Claude Code
- **Security**: security safeguards and best practices